Meeting Note 5

Date: 2026-01-22 **Time:** 10:00 - 11:00 **Location:** PQ703 **Attendees:** LIN Zhanzhi, CAO Yixin (Supervisor)

1. Objectives

- Review the implementation of the operation counting mechanism.
- Discuss the methodology for measuring algorithmic efficiency beyond just wall-clock time.
- Prepare for the constant tuning phase.

2. Progress Report

- **Infrastructure Update**: Implemented `Instrumented<T>` class to intercept and count comparisons, swaps, and assignments during sort execution.
- **Benchmarking Tool**: Updated `count_ops_runner.cpp` to utilize the instrumented class and integrated it into the Python benchmark manager.
- **Validation**: Verified that the instrumented sort behaves identically to the specific sort, ensuring measurement accuracy without altering logic.
- **Preliminary Data**: Collected initial baseline metrics for `std::sort` vs. `Dual-Pivot Quicksort` on random arrays.

3. Discussion & Feedback

Key Discussion Points

- **Metric Significance**: Discussed that while runtime is the ultimate goal, operation counts provide platform-independent insights into algorithmic behavior.
- **Trade-off Analysis**: Acknowledged that `std::sort` (Introsort) minimizes comparisons well, while our Dual-Pivot implementation might trade slightly higher comparisons for better cache locality or fewer swaps.
- **Double vs Int**: Discussed the importance of testing `double` types as comparison costs are higher, potentially altering the optimal constant thresholds.

Supervisor Feedback

> "It is critical that your operation counting does not introduce significant overhead that distorts the relative performance of the branches being taken. Ensure that the 'Instrumented' class is strictly for counting and isn't used for the runtime benchmarks." "When tuning constants, prioritize the reduction of Comparisons over Swaps, as our hardware analysis shows comparisons are significantly more expensive."

4. Issues & Challenges

- **QSort Instrumentation**: `std::qsort` operates on `void*` and raw bytes, making it difficult to instrument assignments or swaps directly.
    - *Status: Resolved* - We will only measure comparisons for `qsort` using a custom comparator wrapper, and accept that swaps/assignments will be reported as 0.

5. Action Items

- [ ] Run the "Operation Cost Analysis" to quantify exactly how much expensive a Comparison is vs a Swap on the deployment hardware. (Due: 2026-01-24)
- [ ] Begin the "Constant Tuning" phase using the new metrics to optimize `MIN_TRY_MERGE_SIZE`. (Due: 2026-01-26)
- [ ] Tune `MIN_FIRST_RUNS_FACTOR` to optimize for partially sorted inputs. (Due: 2026-01-28)

6. Next Meeting Plan

- **Target Date:** 2026-01-29
- **Focus:** Review of Tuned Constants and Final Benchmark Plan.